Chapter 18 — The Principal Architect: Career, Interviews, and Professional Development

> *"The principal engineer is not the engineer who solves the hardest individual problem. The principal engineer ensures the right problems are being solved, at the right level of abstraction, with tradeoffs made explicit and outcomes measured."*

Chapter Overview

This chapter synthesizes everything in the book into a career framework. It answers the question every senior engineer eventually asks: *what does it actually take to operate at principal level in AI/ML infrastructure?* The answer is not more knowledge — it is a different operating mode.

Sections in this chapter:

- 18.1 What "Principal" Means in AI Infrastructure
- 18.2 The Four Superpowers of the Principal AI Performance Architect
- 18.3 Breadth vs Depth — Positioning for Principal Roles
- 18.4 The STAR+Impact Story Framework
- 18.5 Story Bank — Real STAR+Impact Examples
- 18.6 The Principal-Level Differentiating Question
- 18.7 Professional Communication Strategy
- 18.8 Building the Principal Profile — The Five Moves
- 18.9 System Design Interview Playbook
- 18.10 Worked Example — Design ChatGPT at Production Scale [r5]
- 18.11 Worked Example — Distributed LLM Training System
- 18.12 Reading Roadmap and 12-Month Learning Path
- 18.13 Key Takeaways

18.1 What "Principal" Means in AI Infrastructure

The title is used inconsistently across the industry. In AI infrastructure, it has a specific meaning worth defining precisely.

SENIOR ENGINEER	PRINCIPAL ENGINEER
Optimizes the system	Shapes which system gets built
Solves the problem	Identifies the right problem to solve
Delivers within scope	Defines scope
Reports bottlenecks	Translates bottlenecks into roadmap input
Improves current design	Questions whether current design is correct
Has results	Has impact

The principal engineer requires three dimensions simultaneously:

Technical depth. Root-cause a kernel regression, explain the physics of NVLink congestion, write the NCCL tuning guide, and identify whether a workload is compute-bound or bandwidth-bound from a 30-second roofline plot. Not theoretically — from experience, with numbers.

System breadth. Understand how a GEMM tiling decision interacts with tensor parallelism communication patterns, KV cache memory pressure, and cluster TCO simultaneously. When someone changes a batch size, know which three other system parameters it affects before they ask.

Strategic influence. Translate benchmark findings into hardware architecture recommendations that actually get implemented. Your performance data shapes what silicon the company builds next — not just what software ships this quarter.

The transition from senior to principal is not accumulated domain knowledge. It is a change in orientation — from execution to architecture, from local to system, from answering questions to shaping which questions get asked.

18.2 The Four Superpowers of the Principal AI Performance Architect

Each can be developed deliberately. Each maps to stories you should be building now.

Superpower 1 — Multi-Layer Bottleneck Navigation

The ability to traverse from fleet MFU all the way down to the specific kernel and NCCL configuration causing a bottleneck — and back up to a production-deployable fix — is the rarest skill in the field.

```
FLEET LEVEL
  ↓  "Training cluster at 28% MFU — expected 45%"
JOB LEVEL
  ↓  torch.profiler → 40% wall time is inter-kernel idle gaps
COMMUNICATION LEVEL
  ↓  NCCL_DEBUG=INFO → AllReduce serialized, no backward overlap
TOPOLOGY LEVEL
  ↓  TP AllReduce crossing node boundary at IB latency
     0.5ms × 80 layers × 2 AllReduces = 80ms/step wasted
ROOT CAUSE + FIX
  ↑  Restructure: TP=8 within NVLink domain, PP=4 across nodes
VALIDATION
  ↑  MFU: 28% → 47%  |  Training time: 14 days → 8.4 days
SILICON FEEDBACK
  ↑  Topology analysis feeds interconnect design requirements
```

Superpower 2 — Quantitative Architecture Tradeoff Analysis

Not a vague discussion of pros and cons. A numerical model:

> *"Option A delivers X tokens/dollar at Y watts. Option B delivers X+20% tokens/dollar at Y+15% watts. Given utilization target Z%, break-even is N months."*

This is the language of hardware procurement, roadmap decisions, and capital planning. Prose pros/cons is senior-level thinking. Numbers with units is principal-level thinking.

Superpower 3 — Cross-Domain Translation

Fluency in translating between:

- Silicon language: FLOP/byte arithmetic intensity, HBM bandwidth, thermal envelope
- Software language: kernel launch overhead, operator fusion, CUDA Graphs
- Operations language: P99 latency, GPU utilization, KV cache pressure
- Business language: tokens/dollar, cost per user, TCO, payback period

The ability to translate fluently between all four makes technical findings land with decision-makers at every level of the organization.

Superpower 4 — Silicon-to-Application Feedback Loop

The most impactful work happens at the boundary between software and hardware — where benchmark data informs silicon design for the next generation.

Engineers who formalize this feedback loop, quantify architecture tradeoffs with credible data, and present proposals to chip architecture teams become structurally irreplaceable.

Build this superpower by actively presenting benchmark findings to hardware teams, not just software teams.

18.3 Breadth vs Depth — Positioning for Principal Roles

```
FAILURE MODE 1 — Too narrow:
  Can describe GEMM tiling in exact detail but cannot reason about
  how that tiling decision affects AllReduce timing at cluster scale.
  Signal: "I worked on X layer — the rest was handled by the team."

FAILURE MODE 2 — Too broad:
  Can describe every component of the inference stack but cannot
  produce a specific number from a specific profiling investigation.
  Signal: "We saw significant improvement across the board."

PRINCIPAL POSTURE:
  "My profiling methodology starts at fleet MFU, then Nsight Systems
   to isolate wall-time composition, then Nsight Compute for kernel-level
   arithmetic intensity. In this specific investigation, I found GEMM tile
   size was causing 23% L2 cache miss overhead — here are the counter
   values — and the fix improved throughput 18% validated at job level."
```

The practical rule: **when asked to go deep, choose a topic where you have real profiling data.** Depth without data is memorization. Depth with data is experience. The interviewer cannot distinguish the two until they ask for the numbers.

18.4 The STAR+Impact Story Framework

Principal-level interviews require **STAR+Impact** — not generic STAR. The +Impact component is the change to the system, organization, hardware roadmap, or methodology that resulted from your work.

```
S — SITUATION (be precise about hardware, scale, and the metric)
    "A training cluster of 1,024 H100 GPUs running LLaMA-class 70B model
     at 28% MFU — well below the 45% expected for this hardware and
     model configuration. Each percentage point of MFU represents ~$280K
     of wasted training compute per year."

T — TASK (your specific scope; replace "we" with "I" where accurate)
    "I was tasked with the full profiling investigation — from fleet-level
     symptom identification through root cause to validated fix."

A — ACTION (your methodology; name the tools and the decisions)
    "I used torch.profiler to isolate the wall-time breakdown: 40% of step
     time was inter-kernel idle time. NCCL_DEBUG=INFO revealed AllReduce
     was sequential with backward. Nsys confirmed: NCCL stream appeared
     AFTER the backward pass kernels in the timeline. Root cause: TP=8 was
     configured across two nodes over InfiniBand, not within the NVLink domain.
     80 layers × 2 AllReduces × 5.4ms IB latency = 864ms per step.
     I restructured the parallelism: TP=8 within NVLink, PP=4 across nodes."

R — RESULT (specific, measured, attributed to your action)
    "MFU improved from 28% to 47% on the 1,024-GPU cluster. Training time
     for the 70B run reduced from 14 days to 8.4 days."

+IMPACT (system, org, or silicon change that outlasted this one investigation)
    "The TP placement rule — 'TP must stay within the NVLink domain' — was
     added to the cluster pre-flight checklist. All subsequent training jobs
     follow it. The topology analysis methodology was adopted for a hardware
     interconnect bandwidth sizing proposal to the silicon team."

──────────────────────────────────────────────────────────────────────────

WHY +IMPACT MATTERS:
  Senior candidates have results.
  Principal candidates have impact.
  The result ends when the job ends. The impact persists.
──────────────────────────────────────────────────────────────────────────
```

18.5 Story Bank — Real STAR+Impact Examples

Story 1 — Training Throughput Investigation

S: 1,024 H100 training cluster running 70B model at 28% MFU (expected 45%).

T: Owned the complete profiling investigation and remediation.

A: torch.profiler → 40% wall time in inter-kernel gaps. NCCL_DEBUG=INFO → AllReduce serialized with backward (not overlapped). nsys timeline → NCCL stream appearing AFTER backward kernels. Root cause: TP=8 across two nodes over IB (5.4ms per AllReduce). 80L × 2 × 5.4ms = 864ms per step wasted. Fix: TP=8 within NVLink domain, PP=4.

R: MFU 28% → 47%. Training time 14 days → 8.4 days.

+Impact: TP placement rule added to cluster pre-flight checklist. Applies to every subsequent training job. Topology analysis methodology feeds hardware interconnect proposal.

Story 2 — Inference KV Cache OOM Investigation

S: Production LLM serving cluster for 70B model. TTFT P99 spiking to 8+ seconds at 60% KV utilization (expected degradation onset at 85%+). SLA: P99 < 800ms.

T: Identify why degradation was occurring 25 percentage points earlier than expected.

A: Instrumented vLLM with per-request KV page allocation traces. Found that long-context requests (32K tokens) were fragmenting the KV page pool. Even at 60% aggregate KV utilization, the contiguous page availability for new short requests was exhausted. The production traffic was bimodal: 70% short (512 tokens) and 30% long (32K tokens). Page size=16 tokens → too many fragmented partial pages. Implemented: prefix caching for shared system prompt (70% of requests), page size increased from 16 to 32 tokens, dedicated long-context tier.

R: P99 TTFT degradation onset shifted from 60% KV utilization to 81%. Effective serving capacity (requests/GPU/hour at SLA) increased 34%.

+Impact: Fragmentation analysis incorporated into capacity planning model. Prefix cache hit rate became tracked fleet metric on the Prometheus/Grafana dashboard. Policy change: long-context requests routed to separate tier by default.

Story 3 — Data Pipeline Investigation (Training MFU Variance)

S: 512-GPU training run at 42% MFU with unusual step-time variance (some steps 3.2s, others 1.4s). Communication and compute both appeared healthy in existing monitoring.

T: Diagnose root cause of step-time variance. Existing observability stack had no data-loading telemetry.

A: Added InstrumentedDataLoader (5 lines of Python) to measure per-step wait time. Found: 85ms average DataLoader wait per step. Profiled Lustre I/O with fio → read bandwidth 1.8 GB/s (expected 40+ GB/s). Root cause: dataset directory had default stripe_count=1 (all reads going to

one OST out of 48). Set lfs setstripe -c -1 on dataset directory, re-staged data. Added DataLoader metrics to the training Prometheus exporter.

R: DataLoader wait dropped from 85ms to <5ms per step. Step-time variance eliminated. MFU improved from 42% to 53%.

+Impact: Lustre stripe configuration added to cluster onboarding checklist. DataLoader wait time became a standard training telemetry metric logged by default.

18.6 The Principal-Level Differentiating Question

There is one question that separates principal from senior candidates in AI infrastructure interviews more consistently than any other:

> **"Tell me about a time your benchmark findings changed a hardware architecture decision."**

If you have this story — with quantified benchmark data, a specific architectural change made in response, and a measurable production or silicon outcome — you are communicating at principal level. The question tests whether you have crossed the boundary from software optimization to hardware influence. Most candidates have not.

If your stories are only about software fixes to known problems, actively seek opportunities to present benchmark findings to hardware architecture teams. Bring fleet-level MFU data to chip architecture reviews. Quantify what a 20% increase in HBM bandwidth would mean for your
specific workload. Propose a memory subsystem change based on profiling data. This is the work that builds the principal profile — and it requires deliberately stepping outside the boundary of your current role.

18.7 Professional Communication Strategy

Strong written communication reinforces the principal-level impression you build technically.

Rule 1 — Lead with impact, then context:

```
WEAK:  "I've been working on GPU performance for 8 years at Intel and AMD,
        where I worked on everything from mobile to data center systems..."

STRONG: "I improved LLM training throughput 40% on AMD MI300X clusters by
         identifying and fixing AllReduce topology misconfigurations. The
         methodology is now fleet-standard. Here is the technical detail..."
```

Rule 2 — Match the technical vocabulary of the role precisely. "Inference throughput optimization" not "made the model faster." "TTFT P99 at target utilization" not "latency." This signals you operate in their domain, not adjacent to it.

Rule 3 — Quantify ownership explicitly. "I personally owned the profiling analysis and root-cause investigation" versus "our team improved the throughput." Principal-level interviews require attribution clarity. Interviewers evaluate your individual contribution.

Rule 4 — Numbers over adjectives, always. "20% GEMM improvement on the target BERT shape at B=32" communicates more than "significant performance gains." The number forces specificity; the adjective hides the absence of it.

Rule 5 — Be concise. Long messages that bury the point signal unclear thinking, not thoroughness. Use the 3-layer structure:

```
HEADLINE    (15 sec) — The result and why it matters
MECHANISM   (60 sec) — How you achieved it and what you found
IMPLICATION (30 sec) — What it means for the system/org/roadmap
CHECKPOINT           — "Want me to go deeper on any part?"
```

Rule 6 — Name the tradeoff before the solution. "The tension here is X vs Y. I chose X over Y because Z, accepting the cost of W." This framing demonstrates systems thinking and forces you to be explicit about what you gave up — which is what principal engineers are expected to do.

18.8 Building the Principal Profile — The Five Moves

The principal profile is built by accumulating specific types of experience, not by accumulating years. These are the five moves that produce it.

Move	What It Creates	How to Build It
1. Publish a benchmark others reference	You become the authoritative performance source	Benchmark suite. Share findings. Submit to MLPerf. Write the engineering blog post.
2. Change an architecture with data	Your analysis caused hardware or software to change	Profile at system level, not kernel level. Present findings with quantified impact to architecture teams.
3. Build something others depend on	Daily-use infrastructure with your name on it	Performance regression CI. Fleet telemetry dashboards. Capacity planning models.
4. Cross the hardware-software boundary	Work spanning both domains simultaneously	Bring fleet profiling data to silicon teams. Translate FLOP/byte gap into cache sizing requirements.
5. Mentor others into the domain	Leadership multiplier	Write the internal guide. Review profiling analyses. Build the onboarding curriculum. Story: "Engineers I mentored now run these investigations independently."

Build at least three of these five before applying for principal roles.

18.9 System Design Interview Playbook

The Five-Step Structure

```
STEP 1 — CLARIFY REQUIREMENTS (2-3 minutes)
    NEVER jump to solutions. Problem framing is the interview.
    Questions to always ask:
      Model size and architecture?
      SLA targets: TTFT P50/P99? Throughput (tokens/sec)?
      Multi-tenant or single-tenant?
      Streaming responses required?
      Hardware available (H100/H200/MI300X)?
      Scale: current RPS and growth target?
      Budget constraints: latency vs cost optimization?

STEP 2 — HIGH-LEVEL ARCHITECTURE (5-7 minutes)
    Draw the major components. Label data flow AND control plane.
```

```
    For inference: client → gateway → scheduler → runtime → GPU fleet →
observability
    For training:  storage → pipeline → scheduler → framework → comms → GPUs
    Name every box. Unlabeled boxes signal uncertainty.

STEP 3 — NAME THE CENTRAL DESIGN TENSION (3-5 minutes)
    BEFORE proposing solutions, name what makes this system hard.
    "This system must balance TTFT (wants small batches, low queue) vs.
     throughput (wants large batches, high utilization)."
    "This cluster must balance TP efficiency vs. fault isolation."
    Naming the tension before the solution signals architectural thinking.

STEP 4 — BOTTLENECK-DRIVEN DEEP DIVE (3-5 minutes per layer)
    "KV cache pressure is the primary bottleneck here because..."
    Connect every design choice to a measurable constraint.
    Use roofline to classify regime before proposing optimization.
    Back-of-envelope math is mandatory — size the system with numbers.

STEP 5 — BREADTH AWARENESS (2 minutes)
    "If I had more time, I'd design the observability layer —
     specifically ECC/thermal alerting and KV cache pressure dashboards.
     I'd also address the cost model and capacity planning system."
```

Annotations That Score Points

On every diagram, label:

- Performance regime: compute-bound / bandwidth-bound / latency-bound
- Bottleneck resource: KV cache / AllReduce / HBM / PCIe
- Tradeoff: "Prefill: large batch → high compute utilization, higher TTFT"
- Numerical constraint: "Above 82% KV utilization, P99 TTFT degrades sharply"

What Principal-Level Looks Like

```
Minutes 1-5:  PROBLEM FRAMING (not problem solving)
  Candidate asks about model, SLA, hardware, scale.
  Evaluator notes: candidate understands framing is the answer.

Minutes 5-20: ANALYTICAL DECOMPOSITION (not feature enumeration)
  "The core tension here is that prefill is compute-bound and decode is
   memory-bandwidth-bound at the KV access pattern — which means they have
   different optimal batch sizes, different hardware configurations, and
   potentially different serving regimes. The design decision is how to
```

```
   handle that."
  (This framing before solution generation is the principal signature.)

Minutes 20-35: DESIGN WITH QUANTIFIED TRADEOFFS
  "TP=8 within NVLink, not TP=16 across nodes, because NVLink AllReduce
   13GB / 900 GB/s = 14ms, which overlaps with FFN forward at ~14ms.
   Crossing IB: 13GB / 200 GB/s = 65ms — 4× communication overhead."
  (The evaluator needs the number to be grounded, not exact.)

Minutes 35-40: FAILURE MODES AND SCALE
  "At 90% KV utilization, cascading evictions become exponential.
   At 10K GPUs, expected hardware failure rate is ~1.7 hours between events.
   The design handles both with admission control and async checkpointing."

Minutes 40-45: WHAT YOU'D DO WITH MORE TIME
  Shows breadth awareness without claiming completeness.
```

18.10 Worked Example — Design ChatGPT at Production Scale [r5]

> *"The interviewer is not asking whether your design is correct. They are asking whether you understand the system as a physicist understands a problem — from first principles, constraints named, tradeoffs quantified, failure modes anticipated before you are asked."*

Interview prompt: *"Design ChatGPT — training and inference — at production scale."*

Step 0 — Requirements Clarification (Minutes 1-4)

State your assumptions explicitly. Invite correction.

```
Users:         100M daily active users
Peak RPS:      ~50,000 requests/second (estimated from DAU + usage patterns)
Avg context:   2,000 tokens input + 500 tokens output
TTFT SLA:      P50 < 300ms, P99 < 800ms
TPOT SLA:      P50 < 40ms, P99 < 80ms  (15-30 tok/sec = natural reading speed)
Availability:  99.9% uptime
Model:         GPT-class, 175B-1T parameters (MoE), Hopper hardware
Streaming:     Yes — SSE/WebSocket token streaming required
Multi-tenant:  Yes — shared infrastructure, request-level isolation
```

Clarifying questions to ask:

1. Training and inference, or just inference?
2. Latency-optimized (interactive chat) or cost-optimized (batch API)?
3. Global deployment or single region?
4. Budget constraint that determines hardware class?

Step 1 — Back-of-Envelope: Sizing the Inference Fleet

Do this math before drawing anything. State it out loud.

```
INFERENCE FLEET SIZING:

  Model: 70B (representative GPT-class), FP8 quantized
  Weight size: 70 GB → fits on 1× H200 (141 GB) OR 2× H100 with TP=2

  KV cache per request (INT8 KV, S=2K, GQA-8):
    2,048 × 8 heads × 128 d_head × 2 (K+V) × 1B (INT8) × 80 layers = 337 MB

  At 50,000 RPS × 0.337 GB = 16,850 GB of active KV
  On H100 (80GB, 70GB weights, 8GB KV budget): 8/0.337 = 23 concurrent
sessions/GPU
  GPUs for KV at 50K RPS (assuming avg session = 10s, so 500K concurrent):
    500,000 / 23 = ~21,739 GPUs just for KV!

  Wait — recalculate with GQA + FP8 KV:
    FP8 KV: 337 MB → 168 MB per session
    H200 (141GB, 70GB weights, 70GB KV budget): 70/0.168 = 416 concurrent
sessions/GPU
    500K concurrent / 416 = ~1,202 H200 GPUs

  Decode throughput check (H200, 4.8 TB/s HBM):
    TPOT_min = 70 GB / 4.8 TB/s = 14.6ms per token → meets P50 < 40ms SLA ✓
    At batch=416: AI = 1 × 416 = 416 FLOP/byte > 210 ridge point → compute-bound!

  Inference fleet: ~1,500-2,000 H200 SXM GPUs (with redundancy)
  Training fleet (one-time): ~16,384 H100 GPUs for 30-60 days
```

Step 2 — Inference Architecture

```
PRODUCTION INFERENCE SYSTEM:
```

```
USER → CDN / Edge (TLS termination, geographic routing)
     → API Gateway (auth, rate limiting, request logging)
     → Load Balancer (consistent hashing for prefix cache affinity)
     → Serving Layer:

┌──────────────────────────────────────────────────────────────┐
│  PREFILL CLUSTER (compute-optimized H100)                    │
│  Input: full prompt (2K tokens)                              │
│  Output: KV cache for all 2K tokens                          │
│  Batch: large (S=2K, B=64 → AI=128 FLOP/byte → compute)|
│  SLA contribution: TTFT component                            │
│  Optimization: chunked prefill, prefix caching               │
└──────────────────────┬───────────────────────────────────────┘
                       │ KV transfer via RDMA (NIXL, ~35 GB/s)
┌──────────────────────▼───────────────────────────────────────┐
│  DECODE CLUSTER (bandwidth-optimized H200)                   │
│  Input: KV cache + last token                                │
│  Output: new token                                           │
│  Batch: continuous batching (200-400 concurrent seqs)        │
│  SLA contribution: TPOT × N_output_tokens                    │
│  Optimization: PagedAttention, FP8 KV, speculative dec       │
└──────────────────────────────────────────────────────────────┘
     → Response streamer (SSE/WebSocket, token-by-token)
     → Observability: TTFT, TPOT, KV %, queue depth

KEY DESIGN TENSIONS (state these before your solution):
  TTFT vs throughput: small batch → low TTFT; large batch → high throughput
  KV headroom vs concurrency: more sessions → less headroom per session
  P-D disaggregation: +30% utilization gain, +5-20ms KV transfer cost
```

Step 3 — Key Design Decisions with Quantified Tradeoffs

```
DECISION 1: FP8 weights + FP8 KV cache
  Cost:    -0.2% MMLU (negligible for chat)
  Benefit: 2× weight throughput + 2× KV capacity
  Numbers: 1,202 H200s → 600 H200s (same capacity at half the fleet)
  Decision: ALWAYS deploy FP8 in production. Zero reason not to.

DECISION 2: GQA (H_kv=8 from MHA H_kv=64)
  Cost:    -0.3% quality (negligible for chat)
```

```
  Benefit: 8× KV memory reduction (64 to 8 KV heads)
  Numbers: 337 MB → 42 MB per session; 10K concurrent → 420 H200s (not 21K!)
  Decision: GQA is mandatory for any production deployment.

DECISION 3: Disaggregated Prefill-Decode
  Cost:    KV transfer latency (5-20ms per request)
           Operational complexity (two separate fleets to manage)
  Benefit: Prefill cluster runs at compute-optimal batch → 30% higher GPU util
           Decode cluster runs at BW-optimal steady state → no prefill
interruptions
  Numbers: System throughput +25-35% vs co-located P+D
  Decision: Worth it at > 100 GPU serving fleet. Below that: co-located.

DECISION 4: Prefix caching
  Cost:    KV memory for cached prefixes (~300 MB per unique system prompt)
  Benefit: For 2K-token system prompt at 80% hit rate:
           TTFT drops from 340ms to 34ms (cache hit → only 200-token query
prefill)
           GPU-hours for prefill: -73%
  Numbers: See §11.1.6 worked example — 11× concurrent capacity multiplier.
  Decision: Always enable. Requires sticky routing (session → same server).

DECISION 5: Speculative Decoding (7B draft + 70B target)
  Cost:    +8 GB HBM for draft model, 2-3 hours calibration
  Benefit: 2.5-3.5× TPOT improvement for code/structured output (α=0.85-0.92)
  Numbers: At 50K RPS code traffic: decode capacity equivalent to +70% GPU fleet
  Decision: Enable for API-class workloads (code, SQL). Disable for creative chat.

DECISION 6: Admission Control (KV cache at 82% threshold)
  Cost:    Some requests get HTTP 429 and must retry
  Benefit: Prevents KV cascade failure (E[cascade] = 10× evictions at 90% util)
  Numbers: P99 TTFT at 90% util: 8,200ms. At 82%: 410ms. Hard cliff at 88%.
  Decision: Non-negotiable. Set threshold at 82%, not 90%.
```

Step 4 — Training System Architecture

```
TRAINING ARCHITECTURE (70B model, 16,384× H100, 30-day pretraining):

PARALLELISM CONFIG:
  TP = 8   (within 8-GPU NVLink domain — mandatory)
  PP = 8   (8 pipeline stages × 10 layers each = 80 layers)
```

```
  DP = 256 (16,384 / 64 = 256 data-parallel replicas)
  M  = 64  microbatches → bubble = (8-1)/(7+64) = 9.9% ✓

MEMORY CHECK:
  70B × 16 bytes = 1,120 GB total training state
  With ZeRO-2, TP=8, PP=8: ~8.75 GB model state per GPU → fits ✓
  Activations (checkpointed): ~5.4 GB per micro-batch → fits ✓

COMMUNICATION:
  TP AllReduce (per layer, NVLink): 268 MB / 900 GB/s = 0.3ms — overlapped ✓
  PP activation send (per microbatch, IB): 134 MB / 50 GB/s = 2.7ms
  DP gradient sync (ZeRO-2, per step, IB): overlapped with backward ✓

FAULT TOLERANCE:
  Failure rate: 16,384 × 1%/week × (30/7) weeks = 702 failures expected
  Async checkpoint every 200 steps (560 seconds): CPU snapshot in 5s, background
write
  Checkpoint overhead: 5 / (200 × 2.8) = 0.9% → acceptable
  Daly optimal: √(2 × 25s × 600s) = 173 second interval → 62 steps ✓ (use 100
steps)

DATA PIPELINE:
  Pre-tokenized, 200 MB shards on Lustre (stripe_count=-1 on dataset dir)
  Sequence packing: 4,096-token windows, cu_seqlens for FlashAttention varlen
  DataLoader: num_workers=4, pin_memory=True, prefetch_factor=4

OBSERVABILITY:
  DCGM → Prometheus → Grafana (fleet GPU health, every 15s)
  Training MFU logged per step → W&B
  Alert: MFU drops > 10% from 30-min rolling baseline → auto nsys trace
```

Step 5 — Bottleneck Reference Table

```
TRAINING BOTTLENECKS — RANKED BY USER IMPACT:

  Bottleneck              Root Cause                  Fix
  ──────────────────────────────────────────────────────────────────────
  Low MFU (< 40%)         TP crossing NVLink boundary  TP=8 within NVLink domain
  Pipeline bubbles        Too few microbatches          M ≥ 4 × PP; interleaved
1F1B
  Checkpoint stall        Synchronous write             Async CPU snapshot +
```

```
background
  Node failure            Hardware ~1/hr at 16K GPUs   Async ckpt + hot-standby
nodes
  Loss divergence         FP16 overflow, bad data       BF16, grad clipping, data
filter
  ──────────────────────────────────────────────────────────────────────────


INFERENCE BOTTLENECKS — RANKED BY USER IMPACT:

  Bottleneck              Root Cause                  Fix
  ──────────────────────────────────────────────────────────────────────────
  TTFT P99 > SLA           Queue backlog, long prefill  Chunked prefill; prefix
cache
  KV OOM / cascade         KV > 82% utilization        Admission control at 82%
  Slow decode              BW-bound, small batch        Continuous batching;
quantize
  GPU underutil            Overhead-bound               CUDA Graphs; torch.compile
  Prefix cache miss        No sticky routing            Session-ID based LB
routing
  Spec decode rejection    Wrong draft model            Workload-adaptive spec
decode
  ──────────────────────────────────────────────────────────────────────────
```

18.11 Worked Example — Distributed LLM Training System

Interview prompt: *"Design the training infrastructure for a 70B LLM from scratch."*

This walkthrough is condensed from the full treatment in Chapter 10. Use the §18.10 method: clarify requirements → back-of-envelope → architecture → tensions → decisions.

```
REQUIREMENTS CLARIFICATION:
  Model: 70B (given)
  Hardware budget: 512× H100 (assume)
  Target training throughput: MFU ≥ 45%
  Dataset: 1T tokens, web + code + books
  Training duration: within 30 days (cost constraint)

BACK-OF-ENVELOPE:
  FLOPs needed: 6 × 70B params × 1T tokens = 420 ZettaFLOPs
  H100 BF16 effective: 989T × 0.50 MFU = 495 TFLOPS practical
  512 GPUs × 495 TFLOPS = 253 PetaFLOPS
  Time: 420 × 10^21 / 253 × 10^15 = 1.66M seconds = 19.2 days ✓

PARALLELISM CONFIG (512 GPUs, 64 nodes):
  TP=8  (within NVLink), PP=4  (cross-node), DP=16  (gradient sync)
  Bubble: (4-1)/(3+32) = 8.6% ✓ (M=32 microbatches)
  Global batch: 16 × 32 × 4 = 2,048 sequences × 4,096 tokens = 8.4M tokens/step

CENTRAL DESIGN TENSIONS:
  1. TP size: larger TP → better memory; larger TP → must stay within NVLink
  2. Checkpoint frequency: more frequent → less lost work; too frequent → I/O
overhead
  3. Gradient overlap: full overlap → best MFU; requires correct DDP bucket config

KEY DECISIONS:
  FP8 via Transformer Engine → +2× GEMM throughput, zero code change, -0.2%
quality
  ZeRO-2 via FSDP SHARD_GRAD_OP → reduces optimizer state 5.6× vs DDP
  Async checkpointing every 200 steps → 0.9% overhead vs 5%+ synchronous
  Sequence packing → eliminate 84% padding waste → effectively +20% throughput
```

18.12 Reading Roadmap and 12-Month Learning Path

```
12-MONTH PRINCIPAL-LEVEL LEARNING PATH:

  Stage        Months  Focus                          Target Capability
  ──────────────────────────────────────────────────────────────────────
  Foundation   1-2     Chapters 1-5, key papers      Roofline, transformer ops,
                                                      GPU memory hierarchy — cold
  Systems Depth 3-4   Chapters 6-10, experiments    Design distributed training
                                                      from first principles; size
```

```
                                                        cluster with numbers
  Tooling      5-6    Nsight + vLLM + DCGM          Profile inference end-to-end;
  Fluency               hands-on                     identify primary bottleneck
  Architecture 7-9    Chapters 11-17, design        Design, cost, justify a
  Thinking              practice                     training or inference system
  Principal    10-12   Publish, present, mentor      Answer "tell me about a time
  Profile               to hardware teams            your benchmark changed an
                                                      architecture decision"
  ──────────────────────────────────────────────────────────────────────────

ONE WEEK BEFORE INTERVIEW — READING ORDER:
  Day 1: Chapter 18 (this chapter) + review your three STAR+Impact stories
  Day 2: Chapter 11 (KV cache) + Chapter 6 (inference) — most common topics
  Day 3: Chapter 10 (training) + Chapter 14 (networking)
  Day 4: Chapter 12 (benchmarking + roofline) — the diagnostic framework
  Day 5: Chapter 1 (mindset) + Chapter 17 (observability)
  Day 6-7: Rest. One mock system design walkthrough. Trust the preparation.
```

18.13 Key Takeaways

```
IDENTITY
  Principal = technical depth + system breadth + strategic influence.
  All three, simultaneously. Missing any one disqualifies.

THE FOUR SUPERPOWERS
  1. Multi-layer bottleneck navigation — full-stack traversal, not layer expertise
  2. Quantitative tradeoff modeling — numbers, not prose pros/cons
  3. Cross-domain translation — silicon → software → operations → business
  4. Silicon-to-application feedback loop — benchmark data shapes next-gen
hardware

THE DIFFERENTIATING STORY
  "Tell me about a time your benchmark findings changed a hardware architecture
decision."
  Build this story deliberately. It is the single most differentiating narrative
  at principal level. Most candidates do not have it. Be the one who does.

THE FIVE MOVES (build at least 3 before applying):
  1. Publish a benchmark others reference
  2. Change an architecture with data
  3. Build something others depend on
```

```
  4. Cross the hardware-software boundary
  5. Mentor others into the domain

STAR+IMPACT FORMAT
  Senior candidates have results. Principal candidates have impact.
  The +Impact component — the system, org, or silicon change that outlasted
  the investigation — is what registers at principal level.
  Replace "we" with "I" where the "I" is accurate. Evaluators need attribution.

WRITTEN COMMUNICATION
  Specific > vague. Quantified > adjectival. Concise > thorough.
  Attribution-clear > team-attribution. Number first, context second.
  Lead with impact: conclusion first, evidence second.

SYSTEM DESIGN INTERVIEWS
  Never jump to solutions — problem framing IS the interview.
  Name the central tension before proposing the solution.
  Label every tradeoff on every diagram with a number.
  Back-of-envelope math is mandatory — it proves you understand constraints.
  Always connect design choices to measurable outcomes.

CHATGPT-SCALE NUMBERS (know cold for interviews):
  Fleet size (70B inference, 50K RPS): ~1,500–2,000 H200 SXM GPUs
  Training fleet (70B, 1T tokens): ~16,384 H100s for 30 days
  FP8 + GQA + prefix cache combined benefit: up to 11× concurrent capacity
  KV admission control threshold: 82% (not 90% — cascade starts at 88%)
  TP rule: NEVER cross InfiniBand with TP. TP=8 within NVLink. Always.
```

— End of Chapter 18 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect · AMD | Ex-Intel © 2026 — All rights reserved

---

This is the engineer this book was written for. The one who can name the bottleneck before the profiling tool is opened. Who can size the fleet before the whiteboard marker is uncapped. Who can answer "why is decode slower than prefill" with a roofline plot drawn from memory. Who understands that at this scale, every percentage point of MFU is a million dollars. Who has crossed the hardware-software boundary at least once, and remembers what they found.